

A HANDS-ON, CODE-FIRST COURSE

# LangChain From Zero

Build real GenAI apps with Gemini & local models

## PHASE 7 — EMBEDDINGS & VECTOR STORES

---

**Prerequisites:** Phase 6 done — you can load and chunk documents. `faiss-cpu` installed (Phase 0).

**Targets:** embeddings & similarity · GoogleGenerativeAIEmbeddings · OllamaEmbeddings · FAISS vector store

**Estimated time for this phase:** 75–110 minutes

# Where this phase sits

---

You have clean chunks. Now you'll make them **searchable by meaning**: convert each chunk into an **embedding** (a list of numbers), store them in a **FAISS** vector database, and find the chunks closest to any question. This completes RAG's ingestion stage.

**Ph 0–4** ✓ **Setup** → **Fundamentals** → **Models** → **Prompts** → **Chains**

---

**Ph 5** ✓ **Memory & Conversational Apps**

---

**Ph 6** ✓ **Loading & Splitting Documents** — Documents, loaders, chunking.

---

**Ph 7** **Embeddings & Vector Stores** ← you are here  
What embeddings are, creating them (Gemini + Ollama), storing/searching in FAISS.

---

**Ph 8** **Building a RAG Application** — retrieve relevant chunks + answer from them.

---

**Ph 9–12** **Tools & Agents · LangGraph · Deployment · Capstone**

---

# Phase 7 · Embeddings & Vector Stores

## WHAT YOU'LL BE ABLE TO DO AFTER THIS PHASE

Explain what an embedding is and how "similarity" works; generate embeddings with both Gemini ( `GoogleGenerativeAIEmbeddings` ) and a local Ollama model ( `OllamaEmbeddings` ); build a **FAISS** vector store from your chunks; search it by meaning; and save/reload it to disk.

## In this phase

1. 7.1 — The problem: keywords can't find meaning
2. 7.2 — What an embedding actually is
3. 7.3 — Creating embeddings (Gemini & Ollama)
4. 7.4 — Similarity: how "closeness" finds meaning
5. 7.5 — Vector stores & FAISS
6. 7.6 — Build a searchable store from your chunks
7. 7.7 — Saving & reloading the store
8. 7.8 — From store to retriever (Phase 8 preview)
9. 7.9 — Common errors & troubleshooting
10. 7.10 — Mini practice exercise
11. Terms introduced this phase

## 7.1 — The problem: keywords can't find meaning

Imagine a user asks your support bot, "How do I get my money back?" Your handbook chunk says "Refunds are processed within 30 days." A keyword search for "money back" finds *nothing* — none of those words appear. Yet the chunk is exactly the answer.

Embeddings fix this by searching for **meaning**, not words. "Get my money back" and "refunds" land near each other because they *mean* the same thing — even with zero shared words.

## 7.2 — What an embedding actually is

### TERM: EMBEDDING (A VECTOR)

An **embedding** is a list of numbers (a **vector**) that represents the *meaning* of a piece of text. An embedding model reads text and outputs, say, 768 or 3072 numbers. Texts with similar meaning get similar number-lists. The list of numbers is also called a vector; the count of numbers is its **dimensions**.

Think of it as placing every piece of text as a point in a vast "meaning-space." Related ideas cluster together:

```

    meaning-space (simplified to 2-D)

dog •   • puppy
    • cat                               • refund
                                         • money back

    • car   • truck

                                         (far away = unrelated)

```

Real embeddings have hundreds or thousands of dimensions (not 2), but the intuition holds: **close points = similar meaning**. You can't read a vector, but a computer can measure distances between them instantly.

## 7.3 — Creating embeddings (Gemini & Ollama)

An embedding model has two key methods: `embed_query(text)` for a single string, and `embed_documents([texts])` for many at once.

### Gemini — `GoogleGenerativeAIEmbeddings`

```

from dotenv import load_dotenv
from langchain_google_genai import GoogleGenerativeAIEmbeddings

load_dotenv()
embeddings = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001") # 1

vec = embeddings.embed_query("How do I get a refund?") # 2
print(len(vec)) # 3072 (the number of dimensions)
print(vec[:5]) # [-0.024, 0.012, -0.003, -0.057, 0.002] (a peek)

```

### Ollama (local) — `OllamaEmbeddings`

```

from langchain_ollama import OllamaEmbeddings

embeddings = OllamaEmbeddings(model="nomic-embed-text")
# the model you pulled in Phase 0

vec = embeddings.embed_query("How do I get a refund?")
print(len(vec)) # 768 (nomic-embed-text's dimension count – different from Gemini's!)

```

| Line | Explanation                                                                                                                                                                                                                                           |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <code>gemini-embedding-001</code> is Google's current stable text-embedding model. (A newer <code>gemini-embedding-2-preview</code> exists; <code>001</code> is the safe default.) It reads <code>GOOGLE_API_KEY</code> from your <code>.env</code> . |
| 2    | <code>embed_query</code> turns one string into one vector. For many chunks at once, <code>embed_documents([...])</code> returns a list of vectors — far more efficient than looping.                                                                  |
| —    |                                                                                                                                                                                                                                                       |

Note the dimension counts differ (3072 vs 768). That's fine on its own — but it's why you can't mix embedding models, as the next warning explains.

#### THE GOLDEN RULE OF EMBEDDINGS

**Use the SAME embedding model to build your store and to query it.** Vectors from different models live in different spaces with different sizes — comparing them is meaningless and will error or return garbage. Pick Gemini or Ollama for a given store and stick with it. (You can keep two separate stores, one per model.)

## 7.4 — Similarity: how "closeness" finds meaning

#### TERM: COSINE SIMILARITY

To measure how close two vectors are, the usual metric is **cosine similarity**: a score from  $-1$  to  $1$  where **1 = same direction (very similar meaning)**,  $0$  = unrelated,  $-1$  = opposite. You won't compute this by hand — the vector store does it — but that's the math turning "nearby points" into a ranking.

You can see the effect: similar sentences score higher.

```
from langchain_community.utils.math import cosine_similarity

a = embeddings.embed_query("How do I get my money back?")
b = embeddings.embed_query("What is the refund policy?")      # related
c = embeddings.embed_query("How tall is Mount Everest?")     # unrelated

print(cosine_similarity([a], [b]))    # high, e.g. ~0.8
print(cosine_similarity([a], [c]))    # low, e.g. ~0.2
```

That gap — high for related text, low for unrelated — is the entire basis of semantic search.

## 7.5 — Vector stores & FAISS

#### TERM: VECTOR STORE (VECTOR DATABASE)

A **vector store** holds your chunks *and* their embeddings, and is built to answer one question fast: "given this query vector, which stored vectors are closest?" That's how you retrieve relevant chunks in milliseconds, even across millions of them.

We'll use **FAISS** — a free, fast, local vector store from Meta (installed in Phase 0 as `faiss-cpu`). It runs entirely on your machine, no account needed — perfect for learning. (Alternatives like Chroma, Pinecone, or pgvector follow the same LangChain interface, so swapping later is easy.)

## 7.6 — Build a searchable store from your chunks

Now the payoff — the full ingestion pipeline from Phase 6 plus embeddings and storage.

`FAISS.from_documents` takes your chunks and an embedding model, embeds every chunk, and stores them, all in one call.

```
from dotenv import load_dotenv
from langchain_community.document_loaders import TextLoader
from langchain_text_splitters import RecursiveCharacterTextSplitter
from langchain_google_genai import GoogleGenerativeAIEmbeddings
from langchain_community.vectorstores import FAISS # 1

load_dotenv()

# --- Phase 6 steps: load + split ---
docs = TextLoader("sample.txt").load()
chunks = RecursiveCharacterTextSplitter(chunk_size=500,
chunk_overlap=50).split_documents(docs)

# --- Phase 7 steps: embed + store ---
embeddings = GoogleGenerativeAIEmbeddings(model="gemini-embedding-001")
store = FAISS.from_documents(chunks, embeddings)
# 2 embed every chunk + index it

# --- search by meaning ---
hits = store.similarity_search("How do I get a refund?", k=3) # 3 top-3 closest chunks

for i, doc in enumerate(hits, 1):
    print(f"--- hit {i} ---")
    print(doc.page_content[:120])
    print(doc.metadata)
```

| Line | Explanation                                                                                                                                                                                                                                                                         |
|------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 1    | <code>FAISS</code> comes from <code>langchain_community.vectorstores</code> .                                                                                                                                                                                                       |
| 2    | <code>from_documents(chunks, embeddings)</code> does the heavy lifting: it calls the embedding model on every chunk and builds a searchable index. This is the step that costs time/tokens (Gemini) or CPU (Ollama).                                                                |
| 3    | <code>similarity_search(query, k=3)</code> embeds the query with the <i>same</i> model and returns the <code>k</code> closest chunks — as <code>Document</code> objects, metadata intact. Use <code>similarity_search_with_score(...)</code> if you also want the distance numbers. |

For the **local** version, swap two lines — the embedding model, and (optionally) drop `load_dotenv()`:

```
from langchain_ollama import OllamaEmbeddings
embeddings = OllamaEmbeddings(model="nomic-embed-text")
store = FAISS.from_documents(chunks, embeddings) # everything else identical
```

**YOU JUST BUILT SEMANTIC SEARCH**

A query with words that don't appear in any chunk still finds the right chunk, because matching happens in meaning-space. This **store** is the retrieval engine that Phase 8's RAG chain plugs into.

## 7.7 — Saving & reloading the store

Embedding every chunk takes time and (for Gemini) tokens. You don't want to redo it on every run. Save the built store to disk, then load it instantly next time.

```
# save once, after building
store.save_local("faiss_index")          # writes a folder "faiss_index/"

# load later (in another script/run)
store = FAISS.load_local(
    "faiss_index",
    embeddings,                          # MUST be the same embedding model
    allow_dangerous_deserialization=True, # required flag (see note)
)
```

### TWO GOTCHAS WHEN LOADING

- **Same embedding model.** Pass the *same* **embeddings** object you built with. A Gemini-built index can't be queried with Ollama embeddings (golden rule, §7.3).
- **allow\_dangerous\_deserialization=True** is required because loading uses Python's pickle, which can run code. Only set it for index files *you* created and trust — never for a file from an untrusted source.

## 7.8 — From store to retriever (Phase 8 preview)

A vector store can be turned into a **retriever** — a Runnable (Phase 1!) that takes a query string and returns relevant Documents. This is the bridge into RAG: because it's a Runnable, it pipes straight into a chain.

```
retriever = store.as_retriever(search_kwargs={"k": 3}) # return top-3 chunks

docs = retriever.invoke("How do I get a refund?")     # same .invoke as everything
else
print(len(docs))  # 3
```

### TERM: RETRIEVER

A **retriever** is any Runnable that, given a query, returns relevant Documents. A vector store is the most common source, but retrievers are a general interface (some use keyword search, web search, or databases). In Phase 8 you'll pipe this retriever into a prompt + model to get answers grounded in your documents.

## 7.9 — Common errors & troubleshooting

| Symptom                                                | Likely cause & fix                                                                                                                             |
|--------------------------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------|
| Search returns irrelevant or nonsensical chunks        | You likely built with one embedding model and queried with another. Use the same model for build and query (§7.3).                             |
| <code>ModuleNotFoundError: faiss</code>                | <code>pip install faiss-cpu</code> (was in Phase 0; check your venv is active).                                                                |
| Gemini embeddings: 400 / API key error                 | Key not loaded. Confirm <code>load_dotenv()</code> ran and <code>GOOGLE_API_KEY</code> is in <code>.env</code> (Phase 0 §0.10).                |
| Ollama embeddings: connection refused                  | Ollama server not running, or model not pulled. Start Ollama and run <code>ollama pull nomic-embed-text</code> .                               |
| <code>load_local</code> raises a deserialization error | Add <code>allow_dangerous_deserialization=True</code> (for your own trusted index only).                                                       |
| Building the store is very slow                        | Many chunks × per-chunk embedding. Expected, especially locally. Build once and <code>save_local</code> ; reuse with <code>load_local</code> . |
| Gemini: rate-limit errors while embedding many chunks  | Free tier has limits. Embed in smaller batches, or use the local Ollama embeddings for big jobs.                                               |

## 7.10 — Mini practice exercise

### YOUR TURN (DO THIS BEFORE PHASE 8)

**Task:** Take the `ingest.py` from Phase 6 and extend it into `build_store.py`: after splitting, build a FAISS store from the chunks (try it once with Gemini embeddings, once with Ollama), run three `similarity_search` queries — including one whose wording does *not* appear in your document — and print the top hit for each. Then `save_local` the store, and in a separate tiny script `load_local` it and search again to prove persistence works.

**Why:** this builds the exact retrieval engine Phase 8 consumes. The "wording not in the document" query is the one that proves you've got *semantic* search, not keyword matching — the whole point of embeddings.

#### Hints (not the full answer):

- Keep the load+split code from Phase 6 unchanged; you're only adding embed + store + search.
- For the "no shared words" test, paraphrase a fact from your doc using completely different vocabulary.
- Use `similarity_search_with_score` to see the distances and build intuition for what "close" looks like.
- When you reload, remember the same embedding model + `allow_dangerous_deserialization=True`.

## Terms introduced this phase

| Term                          | One-line meaning                                                                                   |
|-------------------------------|----------------------------------------------------------------------------------------------------|
| Embedding / vector            | A list of numbers representing the meaning of text; similar meaning $\Rightarrow$ similar numbers. |
| Dimensions                    | How many numbers are in a vector (e.g. 768, 3072); differs per model.                              |
| embed_query / embed_documents | Embed one string / a list of strings.                                                              |
| Semantic search               | Finding text by meaning rather than matching keywords.                                             |
| Cosine similarity             | A $-1 \dots 1$ score of how aligned two vectors are; higher = more similar.                        |
| Vector store                  | A database of chunks + embeddings built for fast nearest-vector search.                            |
| FAISS                         | A free, fast, local vector store; <code>faiss-cpu</code> .                                         |
| from_documents                | Builds a store by embedding + indexing a list of chunks in one call.                               |
| similarity_search(q, k)       | Returns the <code>k</code> chunks closest in meaning to query <code>q</code> .                     |
| save_local / load_local       | Persist the store to disk and reload it (same model + trust flag).                                 |
| Retriever                     | A Runnable that returns relevant Documents for a query; the bridge to RAG.                         |

### PHASE 7 COMPLETE

You understand embeddings and similarity, can generate vectors with Gemini or a local Ollama model, and can build, search, save, and reload a FAISS vector store — then expose it as a retriever. RAG's ingestion stage is fully built. **Next — Phase 8: Building a RAG Application**, where it all comes together: pipe the retriever into a prompt and model with LCEL so the model answers real questions *using your documents* — with sources — on both Gemini and Ollama.